Solid Master End-to-End Functional Test Flow: Complete Shopper Lifecycle

Flow ID: `E2E-STYLE-HAVEN-COMPLETE-01`

Flow Name: Unified Shopper Journey: Onboarding, Catalog Discovery, Cart Management, Multi-Tier Checkout & Order Fulfillment

Modules Covered:

Module 1: User Accounts & Profile Management

Module 2: Product Catalog, Search, Filtering & Recommendations

Module 3: Shopping Cart (Add/Remove, Quantity Adjustments, Promo, Order Summary)

Module 4: Checkout (Shipping Tier Selection, Payment Gateways, Confirmation)

Primary Objective: Validate the seamless, uninterrupted functional journey of a fashion shopper on Style Haven—from registration and profile setup through search discovery, variant validation, cart mutations, discount application, shipping calculation, multi-method payment execution, order confirmation, and order history tracking.

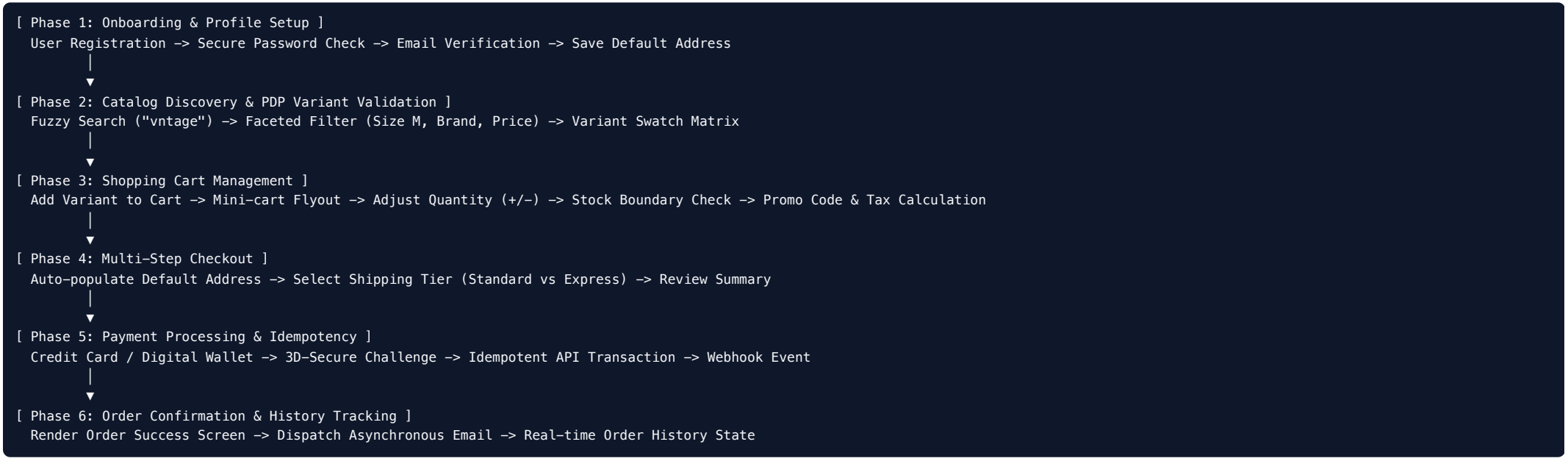
Severity / Priority: P1 / Blocker

Target Execution Environment: Staging / Pre-Production

Estimated Run Time: Manual: ~25 mins | Automated (Playwright): ~1 min 20 sec

---

1. Flow Overview & Architecture Diagram



---

2. Pre-conditions & Test Data Setup

Category	Prerequisite Details
Environment State	Staging / QA environment with active Elasticsearch/Algolia cluster, Redis session/cart store, Stripe sandbox, and notification email worker.
Catalog State	Populated catalog featuring: 1. *Primary Item:* "Linen Oversized Blazer" by Atelier Noir ($120.00). Color: Black, Size: M (Stock = 5). 2. *Secondary Item:* "Silk Slip Dress" by Studio Minimal ($85.00). Color: Emerald, Size: S (Stock = 2).
Promotion Data	Active coupon code: `WELCOME10` (10% off total subtotal). Free shipping threshold: $150.00.
Shipping Matrix	Standard Shipping (3-5 business days): $8.00 (Free over $150). Express Shipping (1-2 business days): $20.00.
Test User Persona	Email: `shopper.fashion.e2e_@stylehaven.io`, Password: `SecurePass#2026!`, Shipping Address: `452 Fashion Ave, Apt 12B, New York, NY 10018`.
Payment Instrument	Stripe 3DS Test Card (`4000 0000 0000 3220`, Exp: 12/28, CVC: 123) for verified frictionless payment; Apple Pay / Digital Wallet mock.

3. Comprehensive Step-by-Step E2E Test Execution

Step #	Stage / Lifecycle Phase	Given / When / Then Test Steps	System Action & Internal Event	Expected Result & Verification Points	Pass/Fail Criteria
01	**User Registration & Policy Enforcement**	**Given** user is on `/register` **When** user inputs first name, last name, unique email, and compliant password `SecurePass#2026!` **And** clicks "Create Account" **Then** registration request is submitted	Client checks password policy (12+ chars, uppercase, lowercase, special char). API triggers `POST /api/v1/auth/register`, hashes password with bcrypt, generates verification token.	- Form validation passes with green check indicators. - Verification screen appears: "Check your inbox". - Account record created in DB with `is_verified: false`.	**PASS:** Verification email dispatched with valid signed token within 5 seconds.
02	**Email Verification & Session Generation**	**Given** verification email is received **When** user clicks the activation link (`/verify?token=...`) **Then** account is activated and user logs in	Token is validated by backend, setting `is_verified: true`. Backend issues `accessToken` (HTTP-only JWT) and `refreshToken`.	- UI displays "Email verified successfully!". - User redirected to `/account/overview`. - Global navbar updates from "Sign In" to user avatar with first name initial.	**PASS:** Authenticated session active; JWT stored securely in HTTP-only cookie.
03	**Profile & Default Address Configuration**	**Given** authenticated user is in `/account/profile` **When** user inputs phone `+1 212-555-0199` **And** adds shipping address: `452 Fashion Ave, Apt 12B, New York, NY, 10018` **And** toggles "Set as Default Shipping Address" **Then** clicks "Save Changes"	Client executes `PUT /api/v1/user/profile` and `POST /api/v1/user/addresses`. Server validates postal code and strips malicious XSS tags.	- Notification toast: "Profile and address saved". - Address card displays "DEFAULT" badge. - Reloading `/account/profile` retains all saved values without data loss.	**PASS:** Address persisted; XSS injection payload sanitized; default flag set to true.
04	**Catalog Discovery & Typo Tolerance**	**Given** user is on storefront **When** user enters typo keyword `vntage jaket` in search bar and presses Enter **Then** search query executes	Request `GET /api/v1/catalog/search?q=vntage+jaket` sent to Elasticsearch/Algolia service. Fuzzy logic matches levenshtein distance <= 2.	- Search header displays: *"Showing results for **vintage jacket** (Did you mean: vintage jacket?)"*. - Results grid populates relevant outerwear items. - Zero-result empty state is NOT triggered.	**PASS:** Accurate results returned despite spelling mistakes; query time < 350ms.

Step #	Stage / Lifecycle Phase	Given / When / Then Test Steps	System Action & Internal Event	Expected Result & Verification Points	Pass/Fail Criteria
05	**Faceted Filtering & Sorting Matrix**	**Given** user views search/catalog results **When** user selects filters: - Category: "Women's Blazers" - Brand: "Atelier Noir" - Size: "M" - Price: "$50 - $150" **And** selects sort: "Newest First" **Then** catalog query refreshes	URL parameters dynamically update to `/catalog?category=blazers&brand=atelier-noir&size=M&price=50-150&sort=newest` without full page refresh.	- Product grid dynamically filters to matching items only. - Filter badge chips display selected attributes with "x" remove options. - Result count counter reflects the filtered subset.	**PASS:** URL state matches applied filters; product grid displays only items satisfying all criteria.
06	**PDP Variant Selection & Out-of-Stock Logic**	**Given** user opens PDP for "Linen Oversized Blazer" **When** user clicks color swatch "Cream" **And** selects size button "M" (Configured out of stock) **Then** stock availability engine runs	Component checks variant matrix `SKU-BLAZER-CRM-M` where `stock_count == 0`.	- "Add to Cart" button immediately changes to disabled **"Out of Stock"**. - Subtext displays "Notify me when available" input box. - Image gallery swaps to photos of the **Cream** color variant.	**PASS:** System prevents adding zero-inventory variant; UI correctly reflects out-of-stock state.
07	**In-Stock Variant Selection & Add to Cart**	**Given** user is on the same PDP **When** user clicks color swatch "Black" **And** clicks size button "M" (Configured in stock = 5) **And** clicks active "Add to Cart" button **Then** cart mutation is triggered	Client issues `POST /api/v1/cart/items` with payload `{ sku: 'SKU-BLAZER-BLK-M', qty: 1 }`. Redis session cart increments.	- Mini-cart drawer flies out from right side of screen. - Product thumbnail, "Black / M", $120.00 price, and quantity '1' display correctly. - Header cart badge icon increments from 0 to 1.	**PASS:** Item added to cart; session cart persists across browser reload.
08	**Secondary Item Addition & Wishlist Navigation**	**Given** user opens second product "Silk Slip Dress" ($85.00) **When** user selects Color: Emerald, Size: S **And** clicks "Add to Cart" **Then** navigates to `/cart` page	Client sends second `POST /api/v1/cart/items`. Cart consolidates items under user ID.	- Full shopping cart page renders 2 distinct line items. - Subtotal displays: $120.00 + $85.00 = **$205.00**. - Free shipping indicator shows "🎉 Congratulations! You unlocked Free Shipping!".	**PASS:** Multiple line items coexist; subtotal calculates accurately.
09	**Cart Quantity Adjustments & Inventory Ceiling**	**Given** user is on `/cart` **When** user clicks `+` on "Silk Slip Dress" (only 2 in stock) **And** clicks `+` a second time (attempting quantity = 3) **Then** inventory boundary validation fires	Client evaluates requested quantity against stock inventory API. Quantity clamps to max available stock (2).	- Quantity updates from 1 to 2; line total updates to $170.00. - Third click triggers inline notification: *"Only 2 units available in stock"*; quantity button disables at 2. - Subtotal recalculates to $120.00 + $170.00 = **$290.00**.	**PASS:** User cannot exceed warehouse inventory ceiling; totals update reactively.
10	**Cart Item Removal & Quantity Reduction**	**Given** user is on `/cart` **When** user clicks `-` on "Silk Slip Dress" reducing it back to 1 unit **And** clicks "Remove" (trash icon) on "Silk Slip Dress" **Then** removal confirmation dialog is accepted	Client sends `DELETE /api/v1/cart/items/{itemId}`. Cart recalculates subtotal with remaining "Linen Oversized Blazer".	- "Silk Slip Dress" is removed with smooth exit animation. - Cart badge decrements to 1. - Subtotal recalculates to **$120.00**. - Free shipping bar re-evaluates: "Add $30.00 more for Free Shipping".	**PASS:** Item cleanly purged from session; UI and price recalculations synchronized.

Step #	Stage / Lifecycle Phase	Given / When / Then Test Steps	System Action & Internal Event	Expected Result & Verification Points	Pass/Fail Criteria
11	**Promo Code & Discount Application**	**Given** cart subtotal is $120.00 **When** user enters coupon code `WELCOME10` into promo box **And** clicks "Apply" **Then** discount engine validates coupon rules	Client sends `POST /api/v1/cart/coupons` with code. Engine checks expiration date, user usage limit, and minimum spend.	- Success toast: "Coupon WELCOME10 applied (10% off)". - Discount line item appears: `-$12.00`. - Discounted subtotal: **$108.00**. - Estimated tax (8.875% NY): **$9.59**. - Order Summary clearly displays: Subtotal ($120.00), Discount (-$12.00), Tax ($9.59).	**PASS:** Discount accurately deducted; taxes calculated on post-discount amount.
12	**Checkout Step 1: Shipping Address Selection**	**Given** user clicks "Proceed to Checkout" **When** user arrives at `/checkout` **Then** step 1 (Shipping Address) renders	Checkout engine queries user profile addresses (`GET /api/v1/user/addresses`).	- Default address saved in Step 03 (`452 Fashion Ave, Apt 12B...`) is pre-selected by default. - Option to "Use a different address" is visible. - Contact phone and email pre-fill automatically.	**PASS:** Zero redundant data entry for logged-in users; address validation passes.
13	**Checkout Step 2: Shipping Method Selection & Live Rate Update**	**Given** shipping address is confirmed **When** user views Shipping Method options: - Standard (3-5 days): $8.00 - Express (1-2 days): $20.00 **And** selects "Express Shipping" **Then** order summary updates reactively	Client sends selected shipping rate ID to checkout session endpoint.	- Order summary immediately recalculates: - Items: $108.00 - Shipping: $20.00 - Tax: $9.59 - **Estimated Total: $137.59**. - Delivery date estimate updates dynamically (e.g., "Arrives by Oct 17").	**PASS:** Shipping fee accurately added to total; UI updates without full page reload.
14	**Checkout Step 3: Payment Gateway & 3D Secure Verification**	**Given** user proceeds to Payment step **When** user inputs test card details with 3DS challenge **And** clicks "Place Order ($137.59)" **And** completes OTP authentication in 3DS modal **Then** payment authorization is processed	Frontend tokenizes card via Stripe Elements; sends `POST /api/v1/orders/checkout` with unique `idempotency_key`. Gateway processes charge.	- 3DS authentication modal resolves successfully. - Loading overlay displays "Securing your order...". - Double-clicking "Place Order" button is prevented via disabled state. - Backend initiates inventory deduction and transitions order to `PAID`.	**PASS:** Payment captured; idempotency prevents double charge; charge ID returned.
15	**Order Confirmation & Email Receipt Delivery**	**Given** payment is authorized **When** browser completes redirect to `/order-success/{orderId}` **Then** order confirmation screen renders	Order service persists order in DB, triggers asynchronous message to email queue (`order.confirmed`), and empties user shopping cart.	- Order Success page displays confirmation checkmark, unique Order ID (e.g., `#SH-98241`), delivery address, and chosen shipping speed. - Shopping cart badge resets to 0. - Customer receives confirmation email with PDF invoice within 30 seconds.	**PASS:** Clean success state; cart cleared; asynchronous email receipt triggered.
16	**Order History & Real-Time Tracking Validation**	**Given** user placed order `#SH-98241` **When** user navigates to `/account/orders` **Then** user inspects order history record	Client fetches `GET /api/v1/user/orders`.	- Most recent order `#SH-98241` appears at top of order history list. - Displays exact purchase date, item thumbnail ("Linen Oversized Blazer - Black / M"), total paid ($137.59), and status badge **"Processing / Order Placed"**. - Clicking "View Order Details" opens complete breakdown with tracking link.	**PASS:** Order persisted in user history; matches checkout invoice and SKU details.

4. Non-Functional, Concurrency & Security Edge Validations

1. **Cart Abandonment & Resumption across Devices:**

User adds "Linen Oversized Blazer" on Mobile Safari, logs into Desktop Chrome 2 hours later. The cart must immediately restore items, quantities, and applied promo code without discrepancy.

2. **Double-Click & Concurrent Checkout Prevention:**

Rapidly clicking "Place Order" 5 times in quick succession must only dispatch one single financial transaction due to frontend button debouncing and backend idempotency keys (`Idempotency-Key:`).

3. **Mid-Checkout Price Alteration:**

If a seller updates the product price while an item sits in a customer's cart, the checkout submission must validate prices against the live database, alert the user of the price difference, and require re-confirmation before charging.

4. **Stock Reservation Timeout:**

When a user enters the final checkout payment step, stock is temporarily held for 10 minutes. If the user abandons the payment modal, the lock expires and stock returns to the public catalog.

---

5. Playwright Automation Implementation Spec

```
// tests/e2e/master-shopper-journey.spec.ts
import { test, expect } from '@playwright/test';

test.describe('Master E2E Flow: Onboarding, Catalog, Cart & Checkout', () => {
  const timestamp = Date.now();
  const testEmail = `shopper_${timestamp}@stylehaven.io`;
  const password = 'SecurePass#2026!';

  test('Complete E2E Shopper Journey from Registration to Paid Order', async ({ page }) => {
    // 1. Registration & Verification
    await page.goto('/register');
    await page.fill('input[name="firstName"]', 'Elena');
    await page.fill('input[name="lastName"]', 'Rostova');
    await page.fill('input[name="email"]', testEmail);
    await page.fill('input[name="password"]', password);
    await page.click('button[type="submit"]');
    await expect(page.locator('.verification-notice')).toBeVisible();

    await page.goto(`/verify?token=mock_valid_token_${timestamp}`);
    await expect(page.locator('.navbar-user-avatar')).toBeVisible();

    // 2. Profile Setup (Address)
    await page.goto('/account/profile');
    await page.fill('input[name="addressLine1"]', '452 Fashion Ave, Apt 12B');
    await page.fill('input[name="city"]', 'New York');
    await page.selectOption('select[name="state"]', 'NY');
    await page.fill('input[name="zip"]', '10018');
    await page.check('input[name="isDefaultShipping"]');
    await page.click('button:has-text("Save Changes")');
    await expect(page.locator('.toast-success')).toBeVisible();

    // 3. Catalog Search, Filter & Variant Selection
    await page.fill('input[type="search"]', 'vntage jaket');
    await page.keyboard.press('Enter');
    await page.click('label:has-text("Atelier Noir")');
    await page.click('button[data-size="M"]');
    await page.click('.product-card:has-text("Linen Oversized Blazer")');

    // Test Out-of-Stock Variant
    await page.click('button[data-color="Cream"]');
    await page.click('button[data-size="M"]');
    await expect(page.locator('button.btn-add-to-cart')).toBeDisabled();

    // Select In-Stock Variant & Add to Cart
    await page.click('button[data-color="Black"]');
    await expect(page.locator('button.btn-add-to-cart')).toBeEnabled();
    await page.click('button.btn-add-to-cart');
    await expect(page.locator('.mini-cart-drawer')).toBeVisible();
    await expect(page.locator('.navbar-cart-badge')).toHaveText('1');
```

```
    // 4. Cart Adjustments & Promo Code
    await page.goto('/cart');
    await expect(page.locator('.cart-line-item')).toHaveCount(1);
    await page.fill('input[name="couponCode"]', 'WELCOME10');
    await page.click('button:has-text("Apply")');
    await expect(page.locator('.discount-line')).toContainText('-$12.00');

    // 5. Checkout: Shipping & Payment
    await page.click('button:has-text("Proceed to Checkout")');
    await expect(page.locator('#selected-shipping-address')).toContainText('452 Fashion Ave');

    await page.click('input[value="express_shipping"]');
    await expect(page.locator('#order-total-amount')).toContainText('$137.59');

    // 6. Complete Payment
    await page.fill('#stripe-card-number', '4000000000003220');
    await page.fill('#stripe-card-exp', '12/28');
    await page.fill('#stripe-card-cvc', '123');
    await page.click('button:has-text("Place Order")');

    // 7. Order Confirmation & Order History
    await expect(page).toHaveURL(/.*order-success.*/);
    const orderId = await page.locator('.order-id-badge').innerText();
    expect(orderId).toContain('#SH-');
    await expect(page.locator('.navbar-cart-badge')).toHaveText('0');

    await page.goto('/account/orders');
    await expect(page.locator('.order-history-list')).toContainText(orderId);
    await expect(page.locator('.order-history-list')).toContainText('$137.59');
  });
});
```